

SCHOOL OF ENGINEERING AND THE BUILT ENVIRONMENT

MATLAB / SIMULINK

Practical Laboratory Manual

Department:	Electrical & Computer Engineering
Module:	Computer Programming I
Level:	Undergraduate (Years 2)
Type:	Lectures, Class Tasks & Self-Study Exercises

Covering: System Modelling | Signal Processing | Control Systems | Dynamic Simulation

TABLE OF CONTENTS

MODULE OVERVIEW	3
PART 1: INTRODUCTION TO SIMULINK	4
1.1 What is Simulink?	4
1.2 The Simulink Environment	5
1.3 Core Building Blocks	6
1.4 Signal Flow and Simulation	7
PART 2: GETTING STARTED — GUIDED WALKTHROUGH	8
2.1 Launching Simulink	8
2.2 Your First Simulation	8
PART 3: ELECTRICAL ENGINEERING MODULES	10
3.1 Tutorial — RC Circuit Transient Analysis	10
3.2 Tutorial — RLC Circuit and Frequency Response	12
3.3 Tutorial — DC Motor Speed Control with PID	14
3.4 Class Task EE-1: RC Transient Response	16
3.5 Class Task EE-2: RLC Band-Pass Filter	17
3.6 Class Task EE-3: PID Motor Control	18
PART 4: MECHANICAL ENGINEERING MODULES	19
4.1 Tutorial — Mass-Spring-Damper System	19
4.2 Tutorial — Vehicle Suspension Model	21
4.3 Tutorial — Thermal System Modelling	23
4.4 Class Task ME-1: Free Vibration Analysis	25
4.5 Class Task ME-2: Suspension Tuning	26
4.6 Class Task ME-3: Thermal PID Control	27
PART 5: SELF-STUDY EXERCISES	28
5.1 Electrical Engineering Self-Study	28
5.2 Mechanical Engineering Self-Study	31
PART 6: ASSESSMENT RUBRIC	34
PART 7: QUICK REFERENCE & TROUBLESHOOTING	35

MODULE OVERVIEW

Purpose and Learning Outcomes

This laboratory manual provides a structured, hands-on introduction to MATLAB/Simulink for students in Electrical Engineering (EE) and Mechanical Engineering (ME). The activities build competence progressively, from understanding the environment to constructing and analyzing real engineering systems.

On completion of this module, students will be able to:

- Build and navigate block diagram models in Simulink
- Simulate dynamic systems in the time and frequency domains
- Apply Simulink to electrical circuits: RC, RLC, and motor control systems
- Apply Simulink to mechanical systems: vibration, suspension, and thermal models
- Interpret simulation outputs using Scope, To Workspace, and Data Inspector tools
- Tune system parameters and analyze the effect on system behavior
- Design basic PID controllers for engineering applications

Structure of This Manual

Section	Description
Part 1: Introduction	Conceptual overview of Simulink — what it is, how it works, core blocks
Part 2: Guided Walkthrough	Step-by-step first simulation — done with the lecturer in class
Part 3: EE Modules	Three EE tutorials + three in-class tasks (RC, RLC, DC Motor PID)
Part 4: ME Modules	Three ME tutorials + three in-class tasks (MSD, Suspension, Thermal)
Part 5: Self-Study	Independent exercises for home study and test preparation
Part 6: Reference	Quick reference tables, keyboard shortcuts, troubleshooting guide

Software Requirements

MATLAB R2021a or later (Student or Academic license)

Simulink toolbox (included with MATLAB)

Control System Toolbox (for PID and transfer function blocks)

Signal Processing Toolbox (recommended for frequency analysis)

All software is available on University lab computers.

Students may also install MATLAB using their student email at: mathworks.com/academia

1.1 What is Simulink?

Simulink is a graphical block diagram environment built on top of MATLAB. It is used for modelling, simulating, and analyzing dynamic systems — systems whose behavior changes over time. Rather than writing equations in code, you build models visually by connecting blocks that represent mathematical operations, physical components, and signals.

Simulink is widely used in industry and research for:

- Control system design (autopilots, motor controllers, thermostats)
- Signal processing (filters, modulation, noise removal)
- Power electronics (inverters, converters, PWM systems)
- Mechanical dynamics (suspension, robotics, structural analysis)
- Embedded systems prototyping (code generation for microcontrollers and FPGAs)
- Aerospace, automotive, biomedical, and chemical engineering applications

Why Simulink? Key Advantages

Feature	Benefit to Engineers
Visual block diagrams	Maps directly to engineering system diagrams — no need to code complex ODEs from scratch
Real-time simulation	See results instantly as parameters change; ideal for control tuning
MATLAB integration	Use MATLAB scripts, functions, and data directly inside Simulink models
Toolbox ecosystem	Specialist toolboxes for control, signal processing, power systems, etc.
Code generation	Automatically generate C/C++ code for embedded systems (e.g. Arduino, STM32)
Industry standard	Widely used by Toyota, Airbus, Tesla, ABB, Siemens, and thousands of engineering companies

Simulink vs. MATLAB Scripts

MATLAB Script	Simulink Model
Text-based code	Graphical block diagram
Best for data analysis and calculations	Best for dynamic system simulation
Runs sequentially	Runs with a continuous-time or discrete-time solver
Difficult to visualize signal flow	Signal flow is visually explicit
Less suited to control design	Purpose-built for control and dynamic modelling

1.2 The Simulink Environment

When you open a new Simulink model, you will see the Simulink Editor — a canvas on which you build your block diagram. The environment has several key areas:

Interface Element	Description
Simulink Canvas (Model Editor)	The white drawing area where blocks are placed and connected with signal lines
Library Browser	A searchable catalogue of all available blocks organized by category
Toolbar	Run, stop, pause simulation; set simulation time; access configuration settings
Simulation Time Box	Specifies how long the simulation runs (e.g. 0 to 10 seconds)
Configuration Parameters (Ctrl+E)	Set solver type, step size, tolerances, and output settings
Diagnostics Viewer	Displays errors and warnings when the model fails to compile or run
Model Explorer	View and manage all variables, signals, and block parameters in the model
Data Inspector	Inspect logged signal data after simulation; compare multiple runs

Navigating the Canvas

- Zoom in/out: Ctrl + scroll wheel, or use the + / - keys
- Pan: Hold spacebar and drag, or middle-click drag
- Fit model to screen: Press Ctrl + Shift + F
- Select all blocks: Ctrl + A
- Search for blocks: Ctrl + Shift + L to open Library Browser; type block name
- Auto-arrange: Select all blocks and press Ctrl + Shift + A for auto-layout

1.3 Core Building Blocks

Simulink models are built from blocks. Each block performs a specific mathematical or logical operation. Blocks are connected by signal lines, which carry numerical values from one block to another.

Block Category	Key Blocks & Their Functions
Sources	Constant, Step, Sine Wave, Ramp, Pulse Generator, Clock — these generate input signals
Sinks	Scope (view waveforms), To Workspace (save data to MATLAB), Display (show current value)
Math Operations	Sum, Gain, Product, Divide, Abs, Trigonometric functions, Math Function
Continuous	Integrator, Derivative, Transfer Fcn (Laplace), State-Space, Zero-Pole
Discrete	Unit Delay, Discrete Transfer Fcn, Discrete Integrator — for sampled/digital systems
Signal Routing	Mux (combine signals), Demux (split signals), Selector, Switch, Bus Creator
User-Defined	MATLAB Function, Interpreted MATLAB Fcn — embed custom MATLAB code in a block
PID Controller	Implements proportional-integral-derivative control with auto-tuning support

Connecting Blocks — Step by Step

1. Add a block: Drag from the Library Browser onto the canvas, or double-click an empty area and type the block name.
2. Connect blocks: Hover over an output port (right side of block) until you see a cross-hair cursor, then drag to an input port.
3. Branch a signal: Hold Ctrl and drag from an existing signal line to create a branch.
4. Label a signal: Double-click on a signal line to add a name label.
5. Delete a connection: Click the line to select it, then press Delete.
6. Flip/rotate a block: Right-click > Format > Flip Block or Rotate Block.

1.4 Signal Flow and Simulation

Simulink models work on the principle of signal flow. At each simulation time step, Simulink evaluates blocks in the correct order (based on how they are connected) and propagates signal values through the model.

The Simulation Solver

Simulink uses a numerical solver to integrate differential equations over time. The solver advances time in steps and computes the state of the system at each step.

- Variable-step solvers (default: ODE45 - Runge-Kutta 4/5): Automatically adjusts step size for accuracy and speed. Best for most continuous systems.
- Fixed-step solvers (e.g. ode4): Uses a constant step size. Required for generating embedded code.

Key Simulation Concepts

Concept	Explanation
Simulation Time	The virtual time over which the model runs (e.g. 0 to 10 s). Set in the toolbar.
Sample Time	How often a block updates its output. Continuous = every solver step. Discrete = fixed rate.
Initial Conditions	Starting state of integrators or state-space blocks. Must be set correctly.
Algebraic Loops	A loop with no delay — cannot be solved directly. Insert a Unit Delay block to break the loop.
Signal Dimensions	Signals can be scalars, vectors, or matrices. Mismatched dimensions cause errors.
Data Types	Default is double (float 64-bit). Can be changed to int, fixed-point, etc. for embedded targets.

Key Concept: The Transfer Function in Simulink

A transfer function $G(s) = Y(s)/U(s)$ represents a linear system in the Laplace domain.

In Simulink: use the Transfer Fcn block. Enter numerator and denominator as coefficient vectors.

Example: $G(s) = 1 / (s^2 + 2s + 1) \Rightarrow$ Numerator: [1] | Denominator: [1, 2, 1]

Coefficients are listed from highest power to lowest power of s .

This is one of the most frequently used blocks in control and circuit analysis.

2.1 Launching Simulink

Follow these steps at the start of every lab session:

7. Open MATLAB by double-clicking its icon (or searching in the Start menu).
8. In the MATLAB Command Window, type: `simulink` and press Enter. Alternatively, click the Simulink button in the MATLAB Home toolbar.
9. The Simulink Start Page opens. Click "Blank Model" to create a new empty model.
10. Save your model immediately: File > Save As > choose your folder > name it meaningfully (e.g. `Lab1_RC_Circuit.slx`). Simulink files use the `.slx` extension.
11. Open the Library Browser: View > Library Browser, or press `Ctrl+Shift+L`.

Important: File Organisation

Create a dedicated folder for each lab (e.g. `Documents\Simulink_Labs\Lab1\`)

Keep your MATLAB workspace and Simulink model files in the SAME folder to avoid path errors.

Never save models to the Desktop or Downloads folder — files may be deleted at logoff.

Use meaningful file names: `Lab1_RC_Circuit.slx`, NOT `untitled.slx` or `model1.slx`

2.2 Your First Simulation — Sine Wave through a Gain

This guided exercise introduces the fundamental workflow: adding blocks, connecting them, setting parameters, and running a simulation. Complete this together with your lecturer.

Objective

Create a model that takes a Sine Wave signal, multiplies it by a constant gain factor, and displays both the original and scaled signals on a Scope.

Step 1: Add a Sine Wave Source

1. In the Library Browser, navigate to: Simulink > Sources
2. Drag the "Sine Wave" block onto the canvas.
3. Double-click the Sine Wave block to open its parameters dialog.
4. Set: Amplitude = 1, Frequency (rad/s) = 2π (this gives 1 Hz), Phase = 0, Bias = 0. Click OK.

Step 2: Add a Gain Block

1. Navigate to: Simulink > Math Operations. Drag the "Gain" block to the right of the Sine Wave block.
2. Double-click the Gain block. Set the Gain value to 2. Click OK.
3. Connect the Sine Wave output to the Gain input by dragging from the ">" output port to the Gain input port.

Step 3: Add a Scope to View Signals

1. Navigate to: Simulink > Sinks. Drag a "Scope" block to the right of the Gain block. Connect Gain output to Scope.
2. To display both signals on one Scope: Add a "Mux" block (Signal Routing > Mux). Set Number of inputs to 2.
3. Connect Sine Wave output to Mux input 1, Gain output to Mux input 2. Connect Mux output to Scope input.

Step 4: Run the Simulation

1. Set simulation stop time to 5 (seconds) in the toolbar text box at the top.
2. Click the green Run button (play triangle) or press Ctrl+T.
3. Double-click the Scope block to open it. You should see two sinusoidal waveforms.
4. Click the Autoscale button (binoculars icon) in the Scope to fit the signals on screen.

Step 5: Explore and Experiment

1. Change the Gain to 0.5 and re-run. Observe the change in the output amplitude.
2. Change the Sine Wave frequency to 4π and re-run. Observe the faster oscillation.
3. Try adding a "Sum" block to add the original and scaled signal together.

Checkpoint — Before Moving On, Verify:

Your model runs without errors (no red highlights on blocks)

The Scope shows two distinct waveforms with different amplitudes

You can change parameters and see the effect on the simulation output

Your model is saved (.slx file) with a meaningful name

3.1 Tutorial — RC Circuit Transient Analysis

Background Theory

A series RC circuit consists of a resistor (R) and capacitor (C) connected in series with a voltage source. When a step voltage is applied, the capacitor charges exponentially. The governing first-order ODE is:

$$RC * dV_c/dt + V_c = V_s(t) \quad \Rightarrow \quad dV_c/dt = (V_s - V_c) / RC$$

The time constant $\tau = RC$ determines how fast the capacitor charges. At $t = \tau$, the capacitor has charged to 63.2% of the source voltage. The circuit reaches steady state after approximately $5 * \tau$.

Transfer Function

In the Laplace domain, the voltage across the capacitor has transfer function:

$$G(s) = V_c(s)/V_s(s) = (1/RC) / (s + 1/RC)$$

This is a first-order low-pass filter with a -3dB corner frequency of $f_c = 1/(2\pi RC)$ Hz.

Building the RC Circuit — Method 1: Transfer Function

Circuit parameters: $R = 1000 \text{ Ohm}$, $C = 100\text{e-}6 \text{ F}$, so $\tau = RC = 0.1 \text{ s}$, $1/RC = 10$

1. Add a Step block: Step time = 0, Initial value = 0, Final value = 5 (representing 5V). Click OK.
2. Add a Transfer Fcn block. Set Numerator = [10], Denominator = [1, 10]. Click OK.
3. Add a Mux block (2 inputs). Connect Step to Mux input 1, Transfer Fcn output to Mux input 2.
4. Add a Scope. Connect Mux output to Scope. Set simulation time to 1 second. Run.
5. Open the Scope. You should see the step input and the exponential charging curve of the capacitor.

Building the RC Circuit — Method 2: Integrator (ODE Form)

This method directly implements the ODE and is more physically intuitive:

1. Add a Step block (input V_s) and a Sum block set to "+-" (plus, minus).
2. Add a Gain block after Sum. Set Gain = $1/RC = 10$. Connect Sum output to Gain.
3. Add an Integrator block (Continuous > Integrator). Set Initial condition = 0. Connect Gain to Integrator.
4. The Integrator output is V_c . Feed V_c back to the minus (-) input of the Sum block. THIS IS THE FEEDBACK LOOP.
5. Add a Scope connected to V_c . Run for 1 second and compare with Method 1 — results should be identical.

Understanding the Feedback Loop

The model implements: $dV_c/dt = (V_s - V_c) / RC$

The Integrator computes the integral of its input, so its output is V_c

V_c is fed back and subtracted from V_s in the Sum block

This fundamental feedback pattern appears in virtually every dynamic system model in Simulink

Correctly implementing feedback is the most critical skill in dynamic system modelling

Expected Results ($R = 1\text{k}\Omega$, $C = 100\mu\text{F}$, $V_s = 5\text{V}$)

Time	Capacitor Voltage (V_c)	Percentage of Final Value
$t = 0.1\text{ s}$ (1 τ)	3.16 V	63.2%
$t = 0.2\text{ s}$ (2 τ)	4.32 V	86.5%
$t = 0.3\text{ s}$ (3 τ)	4.75 V	95.0%
$t = 0.5\text{ s}$ (5 τ)	4.97 V	99.3%
$t \gg \tau$ (steady state)	5.00 V	100%

3.2 Tutorial — RLC Circuit and Frequency Response

Background Theory

A series RLC circuit contains a resistor, inductor (L), and capacitor in series. The transfer function for voltage across the capacitor is:

$$G(s) = (1/LC) / (s^2 + (R/L)s + 1/LC)$$

Key parameters: Natural frequency $\omega_n = 1/\sqrt{LC}$, Damping ratio $\zeta = (R/2)\sqrt{C/L}$

- Underdamped ($\zeta < 1$): Oscillatory response — system rings before settling
- Critically damped ($\zeta = 1$): Fastest non-oscillatory response
- Overdamped ($\zeta > 1$): Sluggish response, no oscillation

Building the RLC Model

Circuit parameters: $R = 100\ \Omega$, $L = 0.01\text{ H}$, $C = 100\text{e-}6\text{ F}$. Calculated: $\omega_n = 1000\text{ rad/s}$, $\zeta = 0.5$ (underdamped)

1. Add a Step source (Step time = 0, Final value = 10V).
2. Add a Transfer Fcn block. Enter: Numerator = [1e6], Denominator = [1, 10000, 1e6]. Click OK.
3. Add Scope. Connect Step > Transfer Fcn > Scope. Run for 0.01 seconds.
4. Observe the underdamped oscillatory response. The capacitor voltage will overshoot the 10V input and oscillate before settling.

Frequency Response Analysis (Bode Plot)

1. In MATLAB Command Window, define the transfer function: `sys = tf([1e6], [1, 10000, 1e6])`
2. Plot the Bode diagram: `bode(sys); grid on`
3. Observe: The magnitude peaks near $\omega_n = 1000$ rad/s, confirming resonance.
4. Use: `[Gm, Pm, Wcg, Wcp] = margin(sys)` to find gain and phase margins.

Varying Damping — Parametric Study

- $R = 20$ Ohm: $\zeta = 0.1$ — heavily underdamped, large oscillations
- $R = 100$ Ohm: $\zeta = 0.5$ — moderate underdamped (default above)
- $R = 200$ Ohm: $\zeta = 1.0$ — critically damped
- $R = 400$ Ohm: $\zeta = 2.0$ — overdamped

Use a MATLAB script: define `R_values = [20, 100, 200, 400]`, loop over them, and plot results on the same graph using `hold on`.

3.3 Tutorial — DC Motor Speed Control with PID

Background Theory

A DC motor is modelled by two coupled equations: the electrical equation (armature circuit) and the mechanical equation (rotor dynamics):

$$\text{Electrical: } L \frac{di}{dt} + R \cdot i = V - K_b \cdot \omega$$

$$\text{Mechanical: } J \frac{d(\omega)}{dt} + B \cdot \omega = K_t \cdot i$$

The simplified transfer function (neglecting armature inductance) from voltage to speed is:

$$G(s) = \frac{\Omega(s)}{V(s)} = \frac{K_t}{((J \cdot s + B) \cdot R + K_t \cdot K_b)}$$

Motor Parameters

Parameter	Symbol	Value
Armature resistance	R	1.0 Ohm
Motor torque constant	K_t	0.01 N.m/A
Back-EMF constant	K_b	0.01 V.s/rad
Moment of inertia	J	0.01 kg.m ²
Friction coefficient	B	0.1 N.m.s/rad

Building the PID Motor Control Model

1. Define motor parameters in MATLAB workspace: $R=1$; $K_t=0.01$; $K_b=0.01$; $J=0.01$; $B=0.1$;
2. Add a Step block (Final value = 100, representing 100 rad/s reference speed).
3. Add a PID Controller block (Continuous > PID Controller). Initially set: $P=1$, $I=0$, $D=0$.
4. Add a Transfer Fcn block for the motor. Numerator = $[K_t/(J \cdot R)]$, Denominator = $[1, (B/J + K_t \cdot K_b/(J \cdot R))]$.
5. Close the control loop: Step > Sum (+/-) > PID > Motor Plant > Scope. Feed Plant output back to negative input of Sum.
6. Run simulation for 5 seconds. Observe the speed response with only P control.

PID Tuning Guide

1. P only ($I=0$, $D=0$): Increase P in steps. Steady-state error decreases but overshoot increases.
2. Add Integral action ($I > 0$): The integrator eliminates steady-state error. Start with $I = 0.5$.
3. Add Derivative action ($D > 0$): Reduces overshoot and improves damping. Try $D = 0.01$ to 0.1 .
4. Use the PID Tuner: In the PID block, click "Tune..." to open the automatic PID Tuner.
5. Target performance: Rise time < 0.5 s, Overshoot < 10%, Steady-state error = 0.

ELECTRICAL ENGINEERING — IN-CLASS TASKS

The following tasks are to be completed in class with lecturer guidance. Work in pairs. Complete each task within the allocated time, and discuss results as a group.

CLASS TASK EE-1: RC Circuit Transient Response Analysis

Objective: Build an RC circuit model in Simulink and investigate how the time constant affects the charging and discharging behaviour.

Steps:

1. Using Method 2 (ODE/Integrator method), build the RC circuit model with $R = 4700 \text{ Ohm}$, $C = 47\text{e-}6 \text{ F}$.
2. Apply a 12V step input at $t = 0$. Calculate the expected time constant τ before simulating.
3. Run the simulation for a duration of at least $5 \cdot \tau$ seconds. Verify your calculated τ against the simulation.
4. Modify the model to simulate discharging: after $5 \cdot \tau$ seconds, reduce the input to 0V using a Pulse Generator.
5. Plot both charging and discharging curves on the same Scope.
6. Create a parametric study: use a MATLAB script to run the simulation for $R = [1000, 4700, 10000] \text{ Ohm}$ with $C = 47 \text{ uF}$. Plot all three responses overlaid using the To Workspace block and MATLAB plot commands.

Deliverables / Expected Outcomes:

- Simulink model (.slx) showing charging and discharging RC circuit
- Scope screenshot showing exponential charge/discharge waveforms
- Table comparing calculated vs simulated time constants for three R values
- Written answer: What is the practical significance of the RC time constant in filter design?

CLASS TASK EE-2: RLC Band-Pass Filter Design

Objective: Design and simulate a series RLC circuit that acts as a band-pass filter centred at a specified frequency.

Steps:

1. Design an RLC band-pass filter with centre frequency $f_0 = 1 \text{ kHz}$ and -3dB bandwidth $BW = 200 \text{ Hz}$.
2. Calculate required L and C values given $R = 50 \text{ Ohm}$. Use: $f_0 = 1/(2 \cdot \pi \cdot \sqrt{LC})$ and $Q = f_0/BW$.
3. Build the Transfer Function model in Simulink for the voltage across the resistor (band-pass output).
4. Apply a sinusoidal input at exactly $f_0 = 1 \text{ kHz}$. Confirm maximum output at this frequency.
5. Test the filter at $f = 500 \text{ Hz}$, 1000 Hz , and 2000 Hz and measure output amplitude at each frequency.
6. Use MATLAB (outside Simulink) to plot the Bode diagram of your filter using `tf()` and `bode()` functions.

Deliverables / Expected Outcomes:

- Simulink model with the correct RLC transfer function
- Frequency response data table (at least 6 frequencies)

- Bode plot (magnitude and phase) showing the band-pass characteristic
- Calculated vs simulated centre frequency — comment on any discrepancy

CLASS TASK EE-3: DC Motor PID Speed Control

Objective: Implement a closed-loop PID speed controller for a DC motor and tune the gains to meet a specified performance requirement.

Steps:

1. Build the complete PID motor control model as described in Tutorial 3.3.
2. First, run the open-loop response (no feedback) with a 12V step input. Record the open-loop steady-state speed.
3. Close the loop with a reference of 150 rad/s. Start with $P=1$, $I=0$, $D=0$.
4. Tune P gain manually: increase P in steps (1, 5, 10, 20). Record overshoot and steady-state error for each.
5. Add integral control to eliminate steady-state error. Record the minimum I required for zero steady-state error.
6. Add derivative control to reduce overshoot to below 5%. Record final PID gains.
7. Use the built-in PID Tuner to find optimal gains automatically. Compare with your manual tuning results.

Deliverables / Expected Outcomes:

- Open-loop response plot with steady-state speed annotation
- Table of P-only results: gain vs overshoot vs steady-state error
- Final PID closed-loop response plot showing reference tracking
- Comparison table: manual PID gains vs auto-tuned gains and performance metrics
- Written reflection (3-4 sentences): What is the practical trade-off between fast response and overshoot in motor control?

4.1 Tutorial — Mass-Spring-Damper System

Background Theory

The Mass-Spring-Damper (MSD) is the archetypal mechanical vibration system. It models a mass m attached to a fixed wall by a spring (stiffness k) and a damper (damping coefficient c). The equation of motion under external force $F(t)$ is:

$$m * \ddot{x} + c * \dot{x} + k * x = F(t)$$

This is directly analogous to the series RLC circuit ($m \sim L$, $c \sim R$, $k \sim 1/C$, $F \sim V$). Key parameters:

- Natural frequency: $\omega_n = \sqrt{k/m}$ rad/s
- Damping ratio: $\zeta = c / (2 * \sqrt{k * m})$
- Damped natural frequency: $\omega_d = \omega_n * \sqrt{1 - \zeta^2}$ (for underdamped systems)

Default System Parameters

Parameter	Symbol	Value	Units
Mass	m	1.0	kg
Spring stiffness	k	100	N/m
Damping coefficient	c	2	N.s/m

Calculated: $\omega_n = 10$ rad/s, $f_n = 1.59$ Hz, $\zeta = 0.1$ (lightly damped — will oscillate)

Building the MSD Model (Two-Integrator Method)

Rewrite as two first-order equations: let $x_1 = x$ (position), $x_2 = dx/dt$ (velocity):

$$\frac{dx_1}{dt} = x_2 \quad \frac{dx_2}{dt} = (F - c * x_2 - k * x_1) / m$$

1. Define in MATLAB workspace: $m=1$; $k=100$; $c=2$;
2. Add Step block (Force input $F = 10$ N, Step time = 0).
3. Add Sum block with 3 inputs: "+- -" (F minus damping force minus spring force).
4. Add Gain block (Gain = $1/m = 1$) after Sum. This gives acceleration.
5. Add first Integrator (IC = 0). Output is velocity ($x_{\dot{}}$).
6. Add second Integrator (IC = 0). Output is position (x).
7. Add Gain block (Gain = $k = 100$). Connect position to this gain. Feed output back to Sum (spring force).
8. Add Gain block (Gain = $c = 2$). Connect velocity to this gain. Feed output back to Sum (damping force).
9. Add Scope to display position x vs time. Run for 5 seconds.

Expected Results (zeta = 0.1, F = 10N step)

- Steady-state displacement: $x_{ss} = F/k = 10/100 = 0.1$ m
- System oscillates with period $T = 2\pi/\omega_d$ approximately 0.63 seconds
- Decaying oscillation — amplitude reduces each cycle due to damping
- Settling time (within 2% of x_{ss}): approximately $4/(\zeta\omega_n) = 4$ seconds

4.2 Tutorial — Vehicle Suspension Model (Quarter-Car)

Background Theory

A quarter-car suspension model represents one wheel of a vehicle. It consists of a sprung mass (vehicle body, m_s) and unsprung mass (wheel assembly, m_u) connected by a spring and damper, sitting on a tyre spring (k_t). Road irregularities excite the system through the tyre.

Equations of motion for the two-DOF quarter-car model:

$$\begin{aligned} m_s \cdot \ddot{x}_s &= -k_s(x_s - x_u) - c_s(\dot{x}_s - \dot{x}_u) \\ m_u \cdot \ddot{x}_u &= k_s(x_s - x_u) + c_s(\dot{x}_s - \dot{x}_u) - k_t(x_u - x_r) \end{aligned}$$

Parameter	Symbol	Value
Sprung mass (body)	m_s	250 kg
Unsprung mass (wheel)	m_u	25 kg
Suspension spring stiffness	k_s	15000 N/m
Suspension damping	c_s	1500 N.s/m
Tyre stiffness	k_t	150000 N/m

Building the Suspension Model

1. Define parameters in MATLAB workspace: $m_s=250$; $m_u=25$; $k_s=15000$; $c_s=1500$; $k_t=150000$;
2. Model the road input x_r using a Sine Wave (amplitude 0.05m, frequency 2 Hz) or Step (pothole).
3. Implement each DOF as a subsystem with Sum, Gain, and two Integrators (velocity and position).
4. Create Subsystem blocks for cleanliness: select related blocks, right-click > Create Subsystem.
5. Output both x_s (body displacement) and x_u (wheel displacement) to a Mux and then Scope.
6. Run for 5 seconds. Observe how the suspension filters road inputs to protect the vehicle body.

Analysis Outputs to Examine

- Body acceleration (differentiate position twice) — affects passenger comfort (ISO 2631 standard)
- Suspension travel ($x_s - x_u$) — must remain within physical limits of suspension stroke
- Tyre deflection ($x_u - x_r$) — wheel must maintain road contact (positive deflection always required)

4.3 Tutorial — Thermal System Modelling

Background Theory

Thermal systems follow the lumped capacitance model. The rate of temperature change depends on heat input and heat loss to the environment:

$$m \cdot C_p \cdot \frac{dT}{dt} = Q_{in} - h \cdot A \cdot (T - T_{amb})$$

This equation is structurally identical to the RC circuit! Thermal time constant: $\tau_{th} = m \cdot C_p / (h \cdot A)$

Thermal Parameter	Electrical Analogue
Temperature T (deg C)	Voltage V (V)
Heat flow Q (W)	Current I (A)
Thermal capacitance $m \cdot C_p$ (J/K)	Electrical capacitance C (F)
Thermal resistance $1/(h \cdot A)$ (K/W)	Electrical resistance R (Ohm)
Ambient temperature T_{amb}	Ground (0 V)

Coffee Cooling Model

Parameter	Value
Mass of liquid	0.25 kg
Specific heat (water)	4186 J/kg.K
Heat transfer coefficient	10 W/m ² .K
Cup surface area	0.02 m ²
Initial temperature	90 deg C
Ambient temperature	20 deg C

1. Calculate thermal time constant: $\tau_{th} = (0.25 \cdot 4186) / (10 \cdot 0.02) = 5232$ s (about 87 minutes).
2. With $Q_{in} = 0$ (cooling only): $dT/dt = -h \cdot A \cdot (T - T_{amb}) / (m \cdot C_p)$
3. Build model: T_{amb} from Constant block, T from Integrator (IC = 90). Sum computes $(T - T_{amb})$. Gain = $-h \cdot A / (m \cdot C_p)$.
4. Run simulation for 10000 seconds. At $t = \tau_{th}$, temperature should be at $90 - 0.632 \cdot (90 - 20) = 45.8$ deg C.

Extension: Thermostat On/Off Control

1. Add a Relay block (Discontinuities > Relay). Switch on: 68, Switch off: 72. Output = 500 W when ON, 0 W when OFF.
2. Feed the heater output as Q_{in} into the thermal model.
3. Observe bang-bang (hysteresis) control — temperature cycles between 68 and 72 deg C.

MECHANICAL ENGINEERING — IN-CLASS TASKS

CLASS TASK ME-1: Free Vibration Analysis of a Mass-Spring-Damper

Objective: Simulate free vibration of the MSD system under different damping conditions and identify the natural frequency and damping ratio from the response.

Steps:

1. Build the MSD model from Tutorial 4.1 with $m = 2$ kg, $k = 200$ N/m.
2. Apply an initial displacement only (set second Integrator IC = 0.05 m, no external force $F = 0$). This is free vibration.
3. Test three damping cases: $c = 1$ (underdamped, $\zeta \sim 0.04$), $c = 40$ (critically damped), $c = 80$ (overdamped). Simulate each for 3 seconds.
4. From the underdamped response, measure the period of oscillation T_d and calculate $\omega_d = 2\pi/T_d$.
5. Apply the logarithmic decrement method: $\delta = \ln(x_1/x_2)$ where x_1 and x_2 are successive peaks. Calculate $\zeta = \delta / \sqrt{4\pi^2 + \delta^2}$.
6. Compare the measured ζ with the calculated value $\zeta = c / (2\sqrt{k \cdot m})$.

Deliverables / Expected Outcomes:

- Three overlaid response plots (underdamped, critically damped, overdamped)
- Table: theoretical vs measured damped frequency and damping ratio
- Full calculation showing the logarithmic decrement method
- Written answer: Why is critical damping important in engineering applications? Give one practical example.

CLASS TASK ME-2: Vehicle Suspension Parameter Study

Objective: Investigate the effect of suspension stiffness and damping on ride comfort and road handling using the quarter-car model.

Steps:

1. Build the full quarter-car suspension model from Tutorial 4.2.
2. Apply a sinusoidal road input of amplitude 0.03 m at frequency 1.5 Hz (typical road undulation).
3. Case 1 (Soft): $k_s = 8000$ N/m, $c_s = 800$ N.s/m. Record body acceleration RMS.
4. Case 2 (Standard): $k_s = 15000$ N/m, $c_s = 1500$ N.s/m (baseline from tutorial).
5. Case 3 (Sport/Stiff): $k_s = 30000$ N/m, $c_s = 3000$ N.s/m.
6. For each case, add a To Workspace block on the body acceleration signal. After simulation, calculate RMS in MATLAB: `rms_val = rms(simout.signals.values)`
7. Discuss the comfort-handling trade-off based on your RMS acceleration values.

Deliverables / Expected Outcomes:

- Three response plots (body + wheel displacement for each case)
- Table of body acceleration RMS values for all three cases
- Written paragraph: Explain the comfort-handling trade-off and how engineers balance it in real vehicles

CLASS TASK ME-3: Thermal System with PID Temperature Control

Objective: Build a thermal model with PID control to regulate an industrial oven to a setpoint temperature.

Steps:

1. Model parameters: Thermal mass $m \cdot C_p = 5000$ J/K, Heat loss $UA = 50$ W/K, Ambient temperature = 25 deg C, Maximum heater power = 2000 W.
2. Build the thermal plant model: $dT/dt = (Q_{\text{heater}} - UA \cdot (T - T_{\text{amb}})) / (m \cdot C_p)$.
3. Add a PID Controller block as the temperature controller with a setpoint (reference) of 200 deg C.
4. Add a Saturation block (0 to 2000 W) to limit heater power to physical bounds.
5. Run open-loop first: apply a constant $Q = 1000$ W and find the steady-state temperature to validate your model.
6. Close the loop. Tune PID gains to achieve: settling time < 300 seconds, overshoot < 5 deg C, zero steady-state error.
7. At $t = 400$ s, introduce a disturbance by changing UA to 100 W/K. Observe the controller response.

Deliverables / Expected Outcomes:

- Simulink model screenshot showing complete closed-loop system
- Temperature response plot including the disturbance rejection at $t = 400$ s
- Final PID gains used and justification for your tuning approach
- Written answer: What is integrator wind-up and why is anti-windup important in this heating system?

PART 5: SELF-STUDY EXERCISES

The following exercises are for independent practice outside of class. They extend the skills developed in tutorials and class tasks. Attempt these before the next lab session and bring your results and questions to class.

5.1 Electrical Engineering Self-Study Exercises

Exercise EE-SS1: High-Pass RC Filter

Problem Statement
A high-pass RC filter outputs voltage across the resistor. Transfer function: $G(s) = RCs / (RCs + 1)$
Given: $R = 10 \text{ k}\Omega$, $C = 10 \text{ nF}$
Tasks:
1. Calculate the -3dB cutoff frequency f_c .
2. Build the Simulink transfer function model for the high-pass filter.
3. Apply a sine wave input and test at $f = 100 \text{ Hz}$, 1 kHz , 10 kHz , 100 kHz .
4. Record output amplitudes and calculate the voltage gain (V_{out}/V_{in}) at each frequency.
5. Plot the Bode magnitude diagram using MATLAB and identify the -3dB point.
6. Modify R to shift the cutoff frequency to 5 kHz . Calculate the new R value required.
Expected: At $f \ll f_c$, output is attenuated. At $f \gg f_c$, output equals input. -3dB at f_c .

Exercise EE-SS2: Op-Amp Inverting Amplifier

Problem Statement
Model an op-amp inverting amplifier in Simulink. Ideal op-amp gain: $A_v = -R_f/R_{in}$ Circuit: $R_{in} = 10 \text{ k}\Omega$, $R_f = 100 \text{ k}\Omega$, so $A_v = -10$
Tasks:
1. Build a Simulink model using a Gain block (gain = -10) to model the ideal op-amp amplifier.
2. Apply a 0.5 V peak-to-peak sine wave at 1 kHz . Verify the output is 5 V peak-to-peak and phase-inverted.

3. Model slew-rate limiting: add a Rate Limiter block (slew rate = 500000 V/s = 0.5 V/us).
4. Apply a 1 MHz square wave and observe slew-rate limiting distortion on the output.
5. Add a Saturation block to limit output to +/- 12V (supply rail limiting). Apply increasing amplitude inputs to observe clipping.

Exercise EE-SS3: Three-Phase Power System Simulation

Problem Statement

Simulate a balanced three-phase AC power system and verify the constant instantaneous power property.

Tasks:

1. Create three Sine Wave blocks: Phase A: amplitude=325V, freq=2*pi*50, phase=0
Phase B: amplitude=325V, freq=2*pi*50, phase=-2*pi/3
Phase C: amplitude=325V, freq=2*pi*50, phase=-4*pi/3
2. Add three resistive loads: R_load = 10 Ohm. Create current signals: $i = V/R$ using Gain blocks.
3. Compute instantaneous power for each phase using Product blocks: $p = v * i$
4. Sum all three phase powers. Observe that total three-phase power is constant (no ripple).
5. Verify: total power = $3 * (V_{rms})^2 / R_{load} = 3 * (325/\sqrt{2})^2 / 10 = 15,844 \text{ W}$
6. Introduce an unbalanced load (Phase C: R = 20 Ohm). Observe the resulting power ripple.

Exercise EE-SS4: Second-Order Filters with Different Damping

Problem Statement

Compare three second-order low-pass filter responses with different damping ratios.

All filters have $\omega_n = 1000 \text{ rad/s}$. $G(s) = \omega_n^2 / (s^2 + 2*\zeta*\omega_n*s + \omega_n^2)$

Tasks:

1. Case A (Underdamped, $\zeta = 0.1$): Numerator=[1e6], Denominator=[1, 200, 1e6]
2. Case B (Butterworth, $\zeta = 0.707$): Numerator=[1e6], Denominator=[1, 1414, 1e6]
3. Case C (Overdamped, $\zeta = 2.0$): Numerator=[1e6], Denominator=[1, 4000, 1e6]
4. Apply a step input to all three simultaneously using a Mux block.
5. Observe differences in rise time, overshoot, and settling time.
6. Use MATLAB to plot Bode diagrams for all three on the same figure.
7. Discussion: Which filter would you choose for audio applications? For control? Justify your choice.

Exercise EE-SS5: PWM LED Dimmer

Problem Statement
Model Pulse Width Modulation (PWM) for LED brightness control.
Tasks:
1. Create a Pulse Generator block: period = 0.001 s (1 kHz), amplitude = 5V, duty cycle = 50%.
2. Add an RC low-pass filter (R=1kOhm, C=100uF) after the PWM to simulate an LED driver filter.
3. Display both the PWM signal and the filtered output on a Scope.
4. Change duty cycle to 25% and 75%. Verify: average filtered output = duty_cycle * 5V.
5. Calculate the peak-to-peak ripple voltage at the filter output. Relate it to the RC time constant.
6. Challenge: Use a Sine Wave controlling a variable duty-cycle source for analogue dimming effect.

5.2 Mechanical Engineering Self-Study Exercises

Exercise ME-SS1: Nonlinear Pendulum Dynamics

Problem Statement
Simulate nonlinear pendulum dynamics and compare with the linearised small-angle model.
Nonlinear ODE: $m \cdot L^2 \cdot \ddot{\theta} + b \cdot \dot{\theta} + m \cdot g \cdot L \cdot \sin(\theta) = 0$
Linearised (small angle): $m \cdot L^2 \cdot \ddot{\theta} + b \cdot \dot{\theta} + m \cdot g \cdot L \cdot \theta = 0$
Parameters: $m=0.5$ kg, $L=1.0$ m, $b=0.1$ N.m.s/rad, $g=9.81$ m/s ²
Tasks:
1. Build the nonlinear model using a Sin Function block for $\sin(\theta)$.
2. Build the linear model using a Gain block for θ .
3. Apply IC = 0.1 rad (small angle). Both models should agree closely.
4. Increase IC to 0.5 rad and 1.0 rad. Observe divergence between models.
5. Compare natural frequencies: $\omega_{n_linear} = \sqrt{g/L}$. Measure actual period from nonlinear model.
6. At what angle does the small-angle approximation introduce more than 5% error?

Exercise ME-SS2: Motorcycle Shock Absorber Design

Problem Statement
Design a shock absorber for a motorcycle front fork using the quarter-car suspension model.
Parameters: Sprung mass=100 kg, Unsprung mass=10 kg, Fork spring $k=20000$ N/m, Tyre $k_t=120000$ N/m
Tasks:
1. Build the two-DOF suspension model.
2. Road input: step bump of 0.08 m height (deep pothole).
3. Test three damper values: $c=500$ (underdamped), $c=2000$ (target design), $c=5000$ (overdamped).
4. For each case, measure: (a) peak body acceleration, (b) maximum suspension travel, (c) settling time.
5. Design criterion: peak body acceleration must be below $1.5g = 14.7 \text{ m/s}^2$.
6. Find the minimum damping value c that satisfies the design criterion.
7. Generate a body acceleration vs damping coefficient graph by running a parametric sweep in MATLAB.

Exercise ME-SS3: Rotating Machinery Imbalance Vibration

Problem Statement
Model vibration caused by a rotating mass imbalance (e.g., unbalanced motor rotor or pump impeller).
Excitation force: $F(t) = m_e \cdot e \cdot \omega^2 \cdot \sin(\omega \cdot t)$
Where: $m_e=0.1$ kg (imbalance mass), $e=0.01$ m (eccentricity), ω = rotational speed (rad/s)
Support structure: $M=50$ kg, $k=500000$ N/m, $c=200$ N.s/m
Tasks:
1. Build the MSD model with the sinusoidal imbalance force as excitation.
2. Run at $\omega=50$ rad/s (below resonance), $\omega=\omega_n$ (at resonance), $\omega=200$ rad/s (above resonance).
3. Calculate ω_n for the support structure. Confirm resonance at this frequency.
4. At resonance, observe how damping limits the peak vibration amplitude.
5. Sweep ω from 10 to 500 rad/s and plot amplitude vs frequency (the frequency response function).

6. Engineering question: Why must rotating machines operate away from natural frequency? What is this condition called?

Exercise ME-SS4: Heat Exchanger Simulation

Problem Statement

Model a simple counterflow heat exchanger using lumped capacitance approach.

Hot fluid: $m_h C_{p_h} = 2000 \text{ J/K}$, $T_{h_in} = 120 \text{ deg C}$ | Cold fluid: $m_c C_{p_c} = 1500 \text{ J/K}$, $T_{c_in} = 20 \text{ deg C}$

Heat transfer between fluids: $Q = UA(T_h - T_c)$ with $UA = 150 \text{ W/K}$

Tasks:

1. Write the two ODEs: $dT_h/dt = (-UA(T_h - T_c))/m_h C_{p_h}$ and similarly for T_c .
2. Build the Simulink model with two integrators (one for T_h , one for T_c).
3. Run to steady state. What are the outlet temperatures? Calculate heat transfer effectiveness.
4. At $t = 100 \text{ s}$, hot fluid inlet temperature drops to 90 deg C . Observe the transient response.
5. Vary UA from 50 to 300 W/K and plot effectiveness vs UA . Discuss for heat exchanger design.

Exercise ME-SS5: Active Vibration Isolation

Problem Statement

Design an active vibration isolation system for a precision optical instrument.

Parameters: Instrument mass = 5 kg , passive spring $k = 1000 \text{ N/m}$, passive damper $c = 20 \text{ N.s/m}$

Active actuator force limit: $\pm 100 \text{ N}$

Tasks:

1. Build the passive MSD model. Apply 0.5 mm , 5 Hz sinusoidal base excitation.
2. Measure the passive transmissibility (ratio of instrument motion to ground motion amplitude).
3. Add an accelerometer model (use two Derivative or two Integrator outputs for velocity/position).
4. Implement skyhook control: $F_{\text{active}} = -C_{\text{sky}} \cdot \dot{x}$ (velocity feedback). Try $C_{\text{sky}} = 50, 200, 500$.
5. Compare passive vs active transmissibility at 5 Hz for each C_{sky} value.

6. Add a Saturation block to enforce the ± 100 N actuator limit. Does this affect performance significantly?

PART 6: ASSESSMENT RUBRIC

Lab Report Assessment Criteria

Each lab report (one per set of class tasks) will be assessed according to the following rubric. Total marks: 100.

Assessment Criterion	Marks	Description
Model Correctness	25	Simulink model builds correctly, runs without errors, signal flow matches the system equations
Simulation Results	25	Results are accurate, compared against theoretical values, discrepancies are explained
Graphical Presentation	15	Scope screenshots and plots are clearly labelled with axes, units, titles, and legends
Analysis and Discussion	20	Students demonstrate understanding of results, identifies trends, answers discussion questions
Parameter Sensitivity	10	Parametric study conducted correctly, effect of changing parameters is correctly interpreted
Professional Presentation	5	Report is well-structured, with clear writing, correct units and significant figures throughout

Submission Requirements

1. Simulink model files (.slx) — all models must be saved with meaningful names
2. MATLAB workspace files (.mat) if To Workspace blocks are used
3. Written lab report in PDF or Word format (minimum 1000 words)
4. All scope screenshots and plots must be embedded in the report (not submitted separately)
5. Submission deadline: End of Week 10
6. Academic integrity: Group discussions are permitted, but all models and reports must be individual work

PART 7: QUICK REFERENCE & TROUBLESHOOTING

Keyboard Shortcuts

Shortcut	Action
Ctrl + T	Run simulation
Ctrl + Shift + T	Stop simulation
Ctrl + E	Open Configuration Parameters
Ctrl + Shift + L	Open Library Browser
Ctrl + Z	Undo
Ctrl + A	Select all blocks
Ctrl + Shift + F	Fit model to screen
Ctrl + Shift + A	Auto-arrange blocks
Ctrl + G	Group selected blocks into subsystem
Ctrl + D	Update diagram (refresh signal dimensions)
Double-click canvas	Quick search and add block by name

Common Blocks Quick Reference

Block Name	Library Path	Common Use
Step	Simulink > Sources	Input step signal at a specified time
Sine Wave	Simulink > Sources	Sinusoidal input signal
Constant	Simulink > Sources	Fixed numerical value
Gain	Simulink > Math Operations	Multiply signal by a constant
Sum	Simulink > Math Operations	Add or subtract signals
Integrator	Simulink > Continuous	Integrate signal over time (1/s in Laplace)
Transfer Fcn	Simulink > Continuous	Implement $H(s) = \text{num}(s)/\text{den}(s)$
PID Controller	Simulink > Continuous	PID control with auto-tuning support
Scope	Simulink > Sinks	Visualise signals during/after simulation
To Workspace	Simulink > Sinks	Export signal data to MATLAB workspace

Mux	Simulink > Signal Routing	Combine multiple signals into a vector
Saturation	Simulink > Discontinuities	Clip signal to upper/lower limits
Relay	Simulink > Discontinuities	On/off (bang-bang) controller with hysteresis
MATLAB Function	Simulink > User-Defined	Embed custom MATLAB code in a block

Troubleshooting Common Errors

Error / Problem	Likely Cause	Solution
Algebraic loop detected	Feedback loop has no integrator or delay	Add a Unit Delay or Memory block in the feedback loop
Signal dimensions do not agree	Blocks have mismatched input/output dimensions	Check all gain/matrix dimensions; use Mux/Demux correctly
Simulation runs but Scope is blank	Scope input port not connected	Verify all signal lines are connected to input ports
Wrong results (but no errors)	Wrong transfer function coefficients	Recheck coefficient ordering — highest power of s first
Integrator output diverges	Unstable system — poles in right-half plane	Check feedback connections — must be negative feedback
Cannot find block in Library	Wrong toolbox or spelling	Press Ctrl+F on canvas to search; check toolbox is installed
Simulation is very slow	Variable-step solver taking tiny steps	Check for near-algebraic loops; reduce output sample rate

Useful MATLAB Commands for Simulink Work

Command	Description
<code>sim('modelName')</code>	Run a Simulink model from the MATLAB command line
<code>tf([num], [den])</code>	Create transfer function object for analysis
<code>bode(sys)</code>	Plot Bode diagram (magnitude and phase vs frequency)
<code>step(sys)</code> <code>pole(sys)</code>	Plot the step response of a transfer function Find poles (roots of denominator) of a transfer function
<code>rlocus(sys)</code>	Plot root locus for closed-loop pole analysis
<code>set_param('model/block', 'Gain', '5')</code>	Set block parameter value programmatically
<code>rms(data)</code>	Calculate the RMS value of a signal vector
<code>lsim(sys, u, t)</code>	Simulate system response to arbitrary input u over time t

Further Learning Resources

- MathWorks Documentation: www.mathworks.com/help/simulink/
- Free Simulink Onramp Course: www.mathworks.com/learn/tutorials/simulink-onramp.html
- Textbook: "Control Systems Engineering" by Norman Nise — excellent theory for control tasks
- Textbook: "Mechanical Vibrations" by Singiresu Rao — covers vibration theory for MSD and suspension
- YouTube: MathWorks official channel has hundreds of free Simulink tutorial videos
- MATLAB Central File Exchange: Community-contributed Simulink models for reference

A Note from the Lecturers

Simulink is a professional engineering tool used daily in industry by engineers at companies such as Toyota, BMW, Airbus, and Eskom.

The skills you develop in these labs will be directly applicable in your careers.

Do not simply copy and run the models — understand WHY each block is there and WHAT it represents physically.

Always check your simulation results against hand calculations or known analytical solutions.

If your results look unexpected, they probably ARE unexpected — trust your engineering intuition and investigate.

Good luck, and enjoy the power of simulation!

— END OF MANUAL —